Déploiement d'applications intelligentes · MLOps

Compte rendu — TP 2

Mettre en œuvre un pipeline MLOps simple

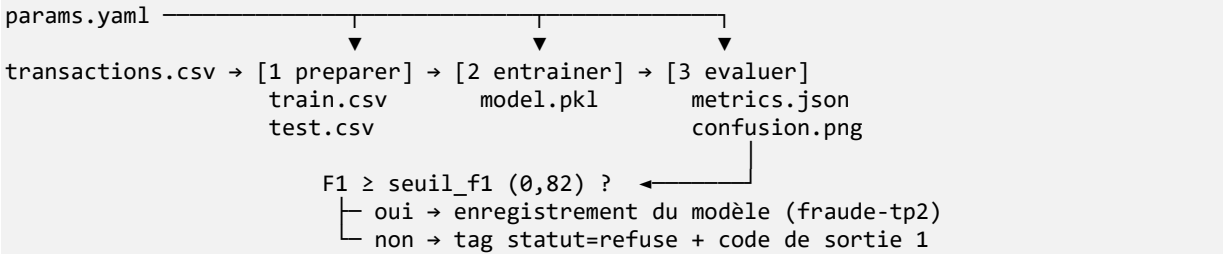
Étudiante : HOUQQA Fatima Zahra · Travail en monôme

Date : 8 octobre 2026

1. Objectif et architecture

Ce TP transforme une suite de commandes en un pipeline en trois étapes (préparation, entraînement, évaluation), piloté par un fichier de paramètres et protégé par une règle de validation : le modèle n'est enregistré dans MLflow que si son F1-score dépasse un seuil.

Schéma du pipeline



Rôle des fichiers

Fichier	Rôle
params.yaml	Tous les réglages (test_size, random_state, n_estimators, max_depth, seuil_f1)
src/preparer.py	Seule étape qui découpe train/test ; écrit data/prepare/train.csv et test.csv
src/entrainer.py	Lit uniquement train.csv ; écrit l'artefact models/model.pkl
src/evaluer.py	Évalue sur test.csv ; écrit reports/metrics.json et confusion.png
pipeline.py	Orchestre les étapes, journalise dans MLflow, applique la règle de validation
MLproject	Rend le pipeline exécutable par mlflow run sur toute machine

2. Réalisation

Le TP a été réalisé sous Windows (PowerShell) ; les commandes de la fiche ont été adaptées (here-strings à la place de cat << EOF, -replace à la place de sed, encodage ASCII pour éviter le BOM qui provoquait un KeyError: 'preparation').

Étape 1 — Externaliser les réglages

Aucune valeur numérique ne reste dans le code. Un outil peut ainsi détecter qu'un réglage a changé, et les paramètres sont journalisés avec log_params.

```
preparation:
  test_size: 0.25
  random_state: 42
entrainement:
  n_estimators: 300
  max_depth: 12
validation:
  seuil_f1: 0.82
```

Étapes 2 à 4 — Préparation, entraînement, évaluation

- **Préparation :** découpage unique avec random_state=42, sans stratify, pour rester identique aux TP 1 et 3. Résultat : 3750 lignes d'entraînement et 1250 lignes de test.
- **Entraînement :** RandomForestClassifier (300 arbres, profondeur 12) entraîné uniquement sur train.csv, sans jamais refaire le découpage.
- **Évaluation :** métriques calculées sur test.csv et écrites dans metrics.json.

Résultats obtenus (identiques à ceux attendus) :

F1	Précision	Rappel
0,8636	0,8769	0,8507

Définition des métriques, avec P la précision et R le rappel :

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2PR}{P + R}$$

Vérification : avec P = 0,8769 et R = 0,8507, on retrouve F1 = 2 × 0,8769 × 0,8507 / (0,8769 + 0,8507) ≈ 0,8636.

Étape 5 — Pipeline et règle de validation

pipeline.py crée une exécution parente « pipeline » et, pour chaque étape, une exécution enfant (nested=True). Les paramètres, les métriques et la matrice de confusion sont journalisés. La règle de validation est :

$$modèle\ enregistré \iff F_1 \geq seuil_{F1} = 0{,}82$$

Le modèle (F1 = 0,8636 ≥ 0,82) a été accepté et enregistré en version 1 de « fraude-tp2 ». L'option skops_trusted_types est nécessaire car MLflow 3.16 enregistre les modèles scikit-learn au format sécurisé skops, qui refuse par défaut les structures internes d'une forêt aléatoire.

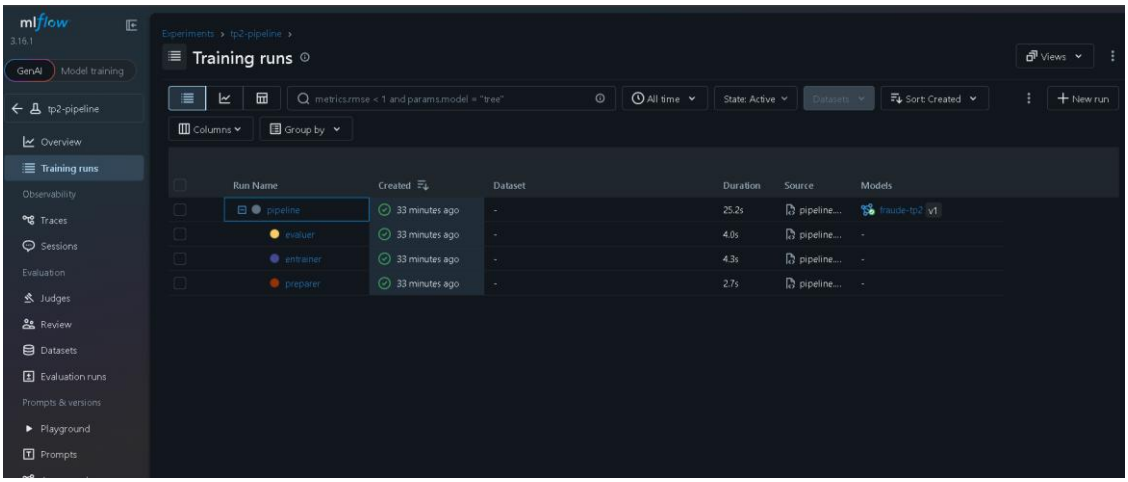


Figure 1 — Run parent « pipeline » et ses trois exécutions enfants (preparer, entrainer, evaluer).

Étape 6 — Vérification du garde-fou

On dégrade volontairement le modèle avec max_depth: 2. Le F1 tombe à 0,2013, sous le seuil : le pipeline lève « modele refuse », renvoie le code de sortie 1, pose le tag statut=refuse, et aucune nouvelle version n'est enregistrée. La valeur max_depth: 12 est ensuite remise.

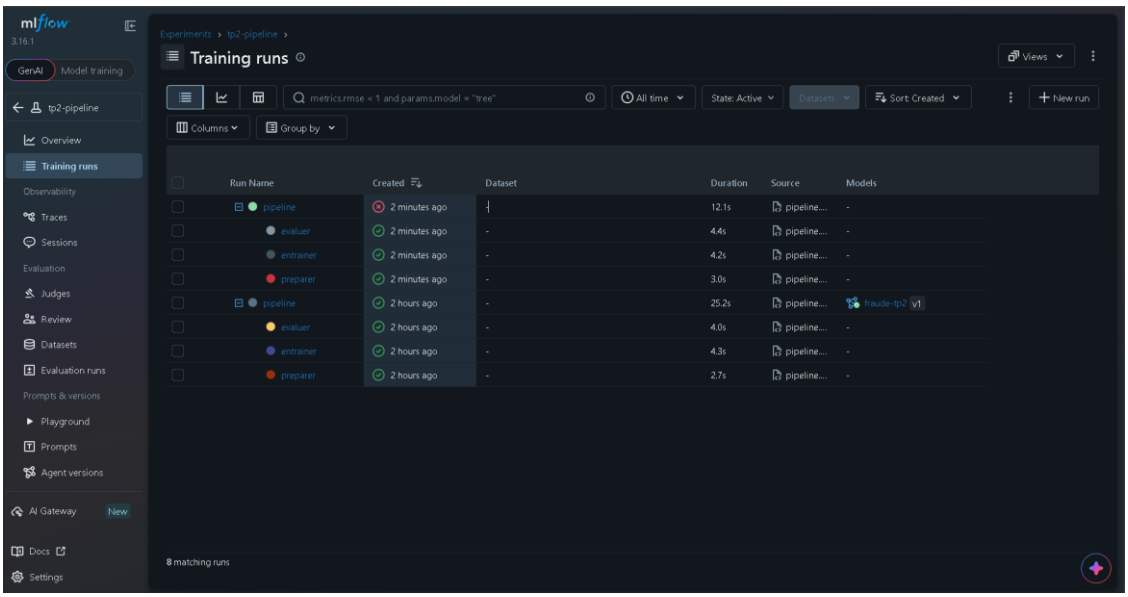


Figure 2 — Run parent refusé (croix rouge, aucun modèle enregistré) au-dessus du run réussi (fraude-tp2 v1).

[Insérer ici la capture de la page du run refusé montrant le tag statut = refuse.]

Étape 7 — Portabilité avec MLproject

Le fichier MLproject déclare le point d'entrée « python pipeline.py ». Le lancement se fait avec le serveur et l'expérience du script :

```
$env:MLFLOW_TRACKING_URI = "http://127.0.0.1:5000"
mlflow run . --env-manager local --experiment-name tp2-pipeline
```

Sans --experiment-name, MLflow refuse (« active experiment ID does not match environment run ID ») ; sans --env-manager local, il tente de recréer un environnement conda. Résultat : [à compléter — « Run succeeded » et version 2 de fraude-tp2].

3. Réponses aux questions

Question 1. Pourquoi l'entraînement ne doit-il jamais refaire le découpage train / test ?

Un nouveau découpage change les lignes de test : des lignes d'évaluation pourraient se retrouver dans l'entraînement (fuite de données), ce qui gonfle artificiellement les scores. Un découpage unique, fait en amont, garantit aussi que tous les modèles sont comparés sur exactement les mêmes lignes.

Question 2. Quel intérêt de structurer les exécutions en parent et enfants ?

Chaque étape a son propre suivi (statut, durée, échec isolé), tandis que le parent regroupe l'ensemble et porte le résultat global : métriques, décision de validation, modèle. L'interface reste lisible et l'on identifie immédiatement l'étape en cause en cas d'échec.

Question 3. Comment choisir le seuil de validation de 0,82 dans un vrai projet ?

À partir du besoin métier (coût d'une fraude manquée comparé à celui d'une fausse alerte), de la performance du modèle actuellement en production ou d'une règle de base, et de la variabilité du F1 (validation croisée, intervalle de confiance). Il est fixé avec les parties prenantes et révisé régulièrement, sans jamais être réglé pour « faire passer » un modèle.

Question 4. Que se passe-t-il si deux étudiants lancent le pipeline avec des params.yaml différents, non versionnés ?

Ils obtiennent des résultats différents sans pouvoir expliquer pourquoi : on ne sait plus quels réglages ont produit quel modèle, et les résultats ne sont pas reproductibles. Il faut versionner params.yaml dans Git et journaliser les paramètres dans MLflow (ce que fait log_params).

Question 5. Quelle étape serait la plus coûteuse sur un vrai jeu de données, et qu'implique-t-elle ?

L'entraînement : il peut durer des heures et coûter cher en calcul. Il faut donc ne le relancer que si ses entrées (données ou paramètres) ont changé (mise en cache, outils comme DVC), séparer les étapes pour ne rejouer que celles concernées, et utiliser un matériel adapté.

Question 6. Avec stratify, le F1 passe de 0,8636 à 0,7849. Le modèle est-il moins bon ?

Non : le modèle n'a pas changé, c'est le jeu de test qui a changé. Le F1 mesure la performance sur un échantillon précis d'environ 1 250 lignes dont seulement une centaine de fraudes (environ 11 %) : peu de positifs, donc une forte variance. Un F1 n'est comparable que sur le même jeu de test. On mesure donc la performance sur un échantillon particulier, avec son incertitude ; il vaut mieux recourir à une validation croisée stratifiée ou à un intervalle de confiance qu'à un seul chiffre.

4. Conclusion

Le pipeline est découpé en trois étapes indépendantes, entièrement paramétré par params.yaml, tracé dans MLflow (parent et enfants), et protégé par une règle de validation dont le refus a été démontré. Il est rejouable en une seule commande avec mlflow run. Les défis bonus n'ont pas été traités dans ce compte rendu.